

Lab 4: Numpy and Matplotlib

CS6.201: Introduction to Software Systems

February 11, 2026

Introduction to Numpy

- Fundamental package for scientific computing with Python.
- Features an N-dimensional array object.
- Provides tools for linear algebra, Fourier transforms, random number capabilities.
- Serves as a building block for other packages (e.g., SciPy, Pandas).
- Works well with plotting libraries (matplotlib, seaborn, plotly etc.).
- Open source.

Why Use NumPy over Python Objects?

- NumPy is implemented in C, making it much faster than native Python lists.
- Uses contiguous memory allocation which improves cache performance.
- Supports vectorized operations (eliminating the need for loops).
- Performs hardware-level optimizations using optimized BLAS and LAPACK libraries.
- More memory-efficient compared to Python lists and tuples.
- Essential for high-performance scientific computing.

Basics

```
1 import numpy as np
2
3 A = np.array([[1, 2, 3],
4               [4, 5, 6]])
5
6 print(A)
7 # Output:
8 # [[1 2 3]
9 #  [4 5 6]]
10
11 Af = np.array([1, 2, 3], dtype=float)
12 # Output:
13 # [1. 2. 3.]
```

Array Creation and Initialization

```
1 np.arange(0, 1, 0.2)
2 # array([0., 0.2, 0.4, 0.6, 0.8])
3
4 np.linspace(0, 2*np.pi, 4)
5 # array ([0. 2.09, 4.18, 6.28])
6
7 A = np.zeros((2, 3))
8 # array([[0., 0., 0.],
9 #        [0., 0., 0.]])
10 # Also available: np.ones, np.diag
11
12 A.shape
13 # (2, 3)
```

Numpy arrays are mutable

```
1 A = np.zeros((2, 2))
2 # array([[ 0.,  0.],
3 #        [ 0.,  0.]])
4
5 C = A # Reference assignment
6 C[0, 0] = 1
7
8 print(A)
9 # [[ 1.  0.]
10 #   [ 0.  0.]]
```

Note: You can prevent this behavior by using the `np.copy()` function to create an independent copy of the array.

Array Attributes

```
1 a = np.arange(10).reshape((2,5))
2
3 print(a.ndim)
4 # 2 (dimensions)
5
6 print(a.shape)
7 # (2, 5)
8
9 print(a.size)
10 # 10 (number of elements)
11
12 print(a.T)
13 # Transpose
14
15 a.dtype
16 # Data type
```

Basic Operations

```
1 a = np.arange(4)
2 # array ([0, 1, 2, 3])
3
4 b = np.array([2,3,2,4])
5
6 print(a * b)
7 # array([0, 3, 4, 12])
8
9 print(b - a)
10 # array([2, 2, 0, 1])
11
12 c = [2, 3, 4, 5]
13 print(a * c) # Works if c is list-like and broadcastable
14 # array([0, 3, 8, 15])
```


Vector Operations

```
1 # Example vectors
2 u = np.array([1, 2, 3])
3 v = np.array([1, 1, 1])
4
5 # Inner Product
6 np.inner(u, v)
7 # Output: 6
8
9 # Outer Product
10 np.outer(u, v)
11 # Output:
12 # array([[1, 1, 1],
13 #        [2, 2, 2],
14 #        [3, 3, 3]])
15
16 # Dot Product (Matrix Multiplication)
17 np.dot(u, v)
18 # Output: 6
```

Matrix Operations (Functions)

- `np.add(A, B)`: Adds matrices A and B element-wise.
- `np.subtract(A, B)`: Subtracts matrix B from matrix A element-wise.
- `np.multiply(A, B)`: Multiplies matrices A and B element-wise.
- `np.divide(A, B)`: Divides matrix A by matrix B element-wise.
- `np.dot(A, B)`: Computes the dot product of matrices A and B.

Matrix Operations (Linear Algebra)

- `A.transpose()` or `A.T`: Transposes matrix `A` (swaps rows and columns).
- `np.matmul(A, B)`: Performs matrix multiplication (supports 2D arrays). Similar to `np.dot` for 2D.
- `np.identity(n)`: Creates an $n \times n$ identity matrix.
- `np.trace(A)`: Computes the trace of matrix `A` (sum of diagonal elements).

Matrix Operations Example

```
1 # Define matrices
2 A = np.ones((3,2))
3 # array([[ 1.,  1.],
4 #        [ 1.,  1.],
5 #        [ 1.,  1.]])
6
7 A.T
8 # array([[ 1.,  1.,  1.],
9 #        [ 1.,  1.,  1.]])
10
11 B = np.ones((2,3))
```

Matrix Operations Example (Continued)

```
1 np.dot(B.T, A.T)
2 # array([[ 2.,  2.,  2.],
3 #        [ 2.,  2.,  2.],
4 #        [ 2.,  2.,  2.]])
5
6 np.dot(A, B.T)
7 # Error: ValueError: shapes (3,2) and (3,2) not aligned:
8 # 2 (dim 1) != 3 (dim 0)
```

1-D Array Slicing Example

```
1 # Generate a 1-D array
2 a = np.array([0.25, 0.56, 0.98, 0.13, 0.72])
3
4 # Select elements from index 2 to the end
5 a[2:]
6 # array([ 0.98, 0.13, 0.72])
7
8 # Select elements from index 1 to 4 (exclusive)
9 a[1:4]
10 # array([ 0.56, 0.98, 0.13])
11
12 # Select every second element
13 a[::2]
14 # array([ 0.25, 0.98, 0.72])
```

1-D Array Slicing Example (Continued)

```
1 # Select elements in reverse order
2 a[::-1]
3 # array([ 0.72, 0.13, 0.98, 0.56, 0.25])
4
5 # Select elements with a negative step from index 4 to 0 (exclusive)
6 a[4:0:-1]
7 # array ([0.13, 0.98, 0.56, 0.25])
8
9 # Select the last element (negative indexing)
10 a[-1]
11 # Output: 0.72
12
13 # Select the second to last element
14 a[-2]
15 # Output: 0.13
```

2-D Array Slicing

```
1 a = np.array([
2     [0.25, 0.56, 0.98, 0.13, 0.72],
3     [0.43, 0.15, 0.67, 0.89, 0.24],
4     [0.91, 0.78, 0.64, 0.38, 0.55],
5     [0.19, 0.82, 0.13, 0.29, 0.71]
6 ])
7
8 # Select the third row, all columns
9 a[2, :]
10 # Output: array([0.91, 0.78, 0.64, 0.38, 0.55])
11
12 # Select the 2nd and 3rd rows, all columns
13 a[1:3]
```


2-D Array Slicing (Continued)

```
1 # Select rows 1 through 3, columns 1 through 4
2 a[1:3, 1:4]
3 # Output: array([[ 0.15,  0.67,  0.89],
4 #               [ 0.78,  0.64,  0.38]])
5
6 # Select rows in reverse order, every other column
7 a[::-1, ::2]
8
9 # Select the last element from the last row
10 a[-1, -1]
11 # Output: 0.71
```

Reshaping Arrays

```
1 # Create a 1D array of size 12
2 a = np.arange(12)
3 print("Original array:", a)
4 # [0 1 2 3 4 5 6 7 8 9 10 11]
5
6 # Reshape the array to 3x4
7 reshaped = a.reshape(3, 4)
8 print("\nReshaped array (3x4):")
9 print(reshaped)
10 # [[ 0  1  2  3]
11 #   [ 4  5  6  7]
12 #   [ 8  9 10 11]]
```

Rules of Reshaping

- The total number of elements in the original array must equal the total number of elements in the reshaped array.
- $OriginalSize = \prod(original_dims) = \prod(new_dims)$
- Example 1: 1D array (12 elements) $\rightarrow 3 \times 4$ array (valid, $3 \times 4 = 12$).
- Example 2: 1D array (12 elements) $\rightarrow 3 \times 5$ array (invalid, $3 \times 5 \neq 12$).

```
1 a = np.arange(12)
2 reshaped = a.reshape(3, 4)           # Valid
3 invalid = a.reshape(3, 5)           # ValueError
```

Random Sampling with NumPy

- For random sampling we use the `np.random` module.

Random Sampling Functions (1)

- `np.random.rand(d0, d1, ..., dn)`: Generates random values from a uniform distribution in range $[0, 1)$ with specified shape.
- `np.random.randn(d0, d1, ..., dn)`: Generates random values from a standard normal distribution (mean=0, variance=1).
- `np.random.randint(lo, hi, size)`: Generates random integers from range $[lo, hi)$.

Random Sampling Functions (2)

- `np.random.choice(a, size, repl, p)`: Randomly samples elements from array `a`.
 - `size`: Number of elements to sample.
 - `repl=True`: Sampling with replacement (same element can be selected multiple times).
 - `repl=False`: Sampling without replacement (each element selected only once).
 - `p`: Probability distribution for sampling.

Understanding Seeds in Random Sampling

- A seed is an initial value used by a pseudorandom number generator (PRNG) to produce a sequence of numbers.
- A seed ensures that random sampling can be reproduced.
- `np.random.seed(seed)`: Sets the seed for the PRNG.

```
1 np.random.seed(42)
2 print(np.random.rand(2,3))
3 # array([[0.37454012, 0.95071431, 0.73199394],
4 #        [0.59865848, 0.15601864, 0.15599452]])
5
6 # Reset seed
7 np.random.seed(42)
8 print(np.random.rand(2,3))
9 # Generates the same numbers
```

Introduction to Masking

- Masking allows for filtering elements of an array based on conditions.
- Uses Boolean expressions to create a mask (True/False).
- Logical operators: & (AND), | (OR).

Masking Examples

```
1 arr = np.array([1, 2, 3, 4, 5, 6])
2
3 # Masking with AND: > 2 AND < 5
4 mask = (arr > 2) & (arr < 5)
5 print(arr[mask]) # Output: [3 4]
6
7 # Masking with OR: < 2 OR > 5
8 mask = (arr < 2) | (arr > 5)
9 print(arr[mask]) # Output: [1 6]
10
11 # Masking with AND: > 2 AND even
12 mask = (arr > 2) & (arr % 2 == 0)
13 print(arr[mask]) # Output: [4 6]
```

Important NumPy Functions (1)

- `np.concatenate()`: Joins arrays along an axis.
- `np.mean()`, `np.median()`, `np.std()`: Statistics.
- `np.unique()`: Finds unique elements.
- `np.split()`: Splits array into sub-arrays.
- `np.argmax()`, `np.argmin()`: Indices of max/min values.

Important NumPy Functions (2)

- `np.argsort()`: Indices that would sort an array.
- `np.hstack()`: Stacks arrays horizontally (column-wise).
- `np.vstack()`: Stacks arrays vertically (row-wise).
- `np.repeat()`: Repeats elements of an array.
- `np.isnan()`: Tests element-wise for NaNs (Not a Number).
- `np.isin()`: Tests whether elements of an array are in another array.
- `np.newaxis()`: Adds a new axis to an array (used for reshaping).

It doesn't end here...

- `np.linalg`: Linear algebra functions (decompositions, solving linear systems, eigenvalues).
- `np.random`: Suite of functions for generating random numbers and probability distributions.
- `np.fft`: Fast Fourier Transform functions for signal processing.

SIMD (Single Instruction Multiple Data)

- Scalar operations process one data point at a time.
- SIMD operations allow a single instruction to operate on multiple data points simultaneously.
- Commonly used in multimedia, graphics, scientific computing, and ML.
- Modern CPUs and GPUs leverage SIMD for faster parallel computations.

Restrictions of SIMD (Self-Study)

- SIMD only works on uniform operations.
- Cannot process different operations on different data elements simultaneously.
- Data must be organized in a **vectorized format**.

Advantages of SIMD (Self-Study)

- Faster computations for large datasets.
- Efficient for repetitive tasks like image and audio processing.
- Reduces the number of instructions executed.
- Boosts performance in machine learning algorithms.

Vectorization

- Vectorization refers to replacing explicit loops with optimized, low-level operations.
- Makes use of SIMD to perform multiple operations simultaneously.
- Eliminates the need for loops in Python.

Advantages of Vectorization (Self-Study)

- Improves speed and memory efficiency.
- Especially useful in deep learning for faster model training.
- Reduces the need for explicit loops, making code more concise.
- Utilizes optimized low-level implementations like BLAS and SIMD.
- Enables parallel execution on modern hardware (GPUs, multi-core CPUs).

Vectorization vs For-Loops

```
1 import numpy as np
2 from time import time
3
4 NO_OF_ELEMENTS = int(1e6)
5 w = np.random.rand(NO_OF_ELEMENTS)
6 x = np.random.rand(NO_OF_ELEMENTS)
7 c = 0
8
9 start_time = time()
10 # Non-vectorized
11 for i in range(NO_OF_ELEMENTS):
12     c += w[i] * x[i]
13
14 # Vectorized (Faster)
15 c = np.dot(w, x)
```

Optimizing Performance in NumPy:

- **SIMD:** Hardware technique for processing multiple data points with a single instruction.
- **Vectorization:** Software technique leveraging NumPy to perform operations on entire arrays without explicit loops.
- **Broadcasting:** Powerful feature enabling operations on arrays of different shapes without reshaping, promoting memory efficiency.

Numpy Summary & Next Steps

- Numpy is powerful for arrays, linear algebra, and random sampling.
- **Next Steps:**
 - Pandas (Data manipulation)
 - SciPy (Advanced scientific computing)
 - Scikit-learn (Machine Learning)
 - PyTorch (Deep Learning)
- **Side Project:** Implement a neural network from scratch using NumPy to deepen your understanding (requires backpropagation).

- A powerful Python library for static, animated, and interactive visualizations.
- Widely used in scientific computing.

Basic Plot

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 4, 5]
4 y = [10, 20, 25, 30, 40]
5
6 plt.plot(x, y, marker='o', linestyle='-', color='b')
7 plt.xlabel('X Axis')
8 plt.ylabel('Y Axis')
9 plt.title('Basic Line Plot')
10 plt.show()
```

Scatter Plot

```
1 x = np.random.rand(50)
2 y = np.random.rand(50)
3 colors = np.random.rand(50)
4
5 plt.scatter(x, y, c=colors, alpha=0.5, cmap='viridis')
6 plt.colorbar()
7 plt.title('Scatter Plot')
8 plt.show()
```

Scatter and Line Plot Combined

```
1 x = np.linspace(0, 10, 100)
2 y = np.sin(x)
3 x_scatter = np.random.rand(30) * 10
4 y_scatter = np.sin(x_scatter) + np.random.randn(30) * 0.1
5
6 plt.plot(x, y, label='Line Plot', color='b')
7 plt.scatter(x_scatter, y_scatter, label='Scatter Plot', color='r')
8
9 plt.legend()
10 plt.show()
```


Bar Plot

```
1 categories = ['A', 'B', 'C', 'D']
2 values = [3, 7, 1, 8]
3
4 plt.bar(categories, values, color=['red', 'blue', 'green', 'purple'])
5 plt.xlabel('Categories')
6 plt.ylabel('Values')
7 plt.title('Bar Chart')
8 plt.show()
9 # Use plt.barh() for horizontal
```

Side by Side Bar Plot

```
1 categories = ['A', 'B', 'C']
2 val1 = [10, 20, 30]
3 val2 = [15, 25, 35]
4 width = 0.25
5
6 r1 = np.arange(len(categories))
7 r2 = [x + width for x in r1]
8
9 plt.bar(r1, val1, color='b', width=width, label='Series 1')
10 plt.bar(r2, val2, color='r', width=width, label='Series 2')
11
12 plt.xticks([r + width/2 for r in r1], categories)
13 plt.legend()
14 plt.show()
```

Histogram

```
1 data = np.random.randn(1000)
2
3 plt.hist(data, bins=30, color='blue', edgecolor='black', alpha=0.7)
4 plt.xlabel('Value')
5 plt.ylabel('Frequency')
6 plt.title('Histogram')
7 plt.show()
```

Subplots

```
1 plt.figure(figsize=(8,6))
2
3 plt.subplot(2, 1, 1)
4 plt.plot([1, 2, 3], [4, 5, 6], 'r')
5 plt.title('First Subplot')
6
7 plt.subplot(2, 1, 2)
8 plt.plot([1, 2, 3], [10, 20, 30], 'b')
9 plt.title('Second Subplot')
10
11 plt.tight_layout()
12 plt.show()
```

Advanced Visualization: subplots()

- **subplot()**: Allows you to manually define grid layout (e.g., 2 rows, 2 columns) and select the plot's position.
- **subplots()**: A more flexible approach, returning a figure and an array of axes, ideal for complex layouts.
- Other plots: Pie Chart, Heatmap, Box Plot.

Questions?

Questions?